

Hybrid Fresnel Harvester — Module Specification

Purpose (one line): A small, input-agnostic module that extracts compact annular/phase features and low-resolution depth/normals from streaming frames (ImageBitmap) at eviction time — enabling both lightweight motion harvesting and deferred high-quality Fresnel-aware image reconstruction.

Overview

This module sits inside a ring-buffer eviction hook. For each incoming frame it produces two classes of outputs:

1. **Realtime / cheap outputs:** a compact *annular energy vector* (K floats) and optional lightweight motion descriptors for immediate motion visualization and event detection.
2. **Deferred reconstruction seeds:** a quantized low-resolution depth map and oct-encoded normal map (e.g., 64×64 or 128×128) plus PSF/optics metadata for later, higher-fidelity Fresnel phase/PSF simulation and super-resolution.

Design goals: small API, input-agnostic, low memory footprint, network/codec tolerant, and easy to drop into MotionPainter’s eviction hook without blocking the main render thread.

Key concepts

- **Annular harvesting:** bin pixels into angular rings (derived from normals or depth) and compute per-ring aggregated signals; this emulates a segmented Fresnel collector in software.
 - **Phase seed (sag map):** integrate low-res normals → depth (sag) and encode a quantized $s(x, y)$ for later conversion to phase $\Phi = 2\pi(n-1)s/\lambda$.
 - **Compact descriptors:** store tiny vectors rather than full frames when possible to keep storage and bandwidth low.
 - **Deferred heavy processing:** heavy depth refinement, PSF computation, and super-resolution happen offline when a frame is flagged.
-

Inputs & assumptions

- **Input type:** `ImageBitmap` (primary). The module accepts `ImageBitmap`, `ImageData`, or a GPU `Texture` proxy. It does not require raw camera metadata, though focal/optics params improve results.
 - **Optional inputs:** precomputed `depth` or `normals` if available from a depth sensor or earlier stage.
 - **Config:** `K` (number of annular bins), `lowRes` (resolution for depth/normals), quantization bit depths, expected `z_near / z_far`, `lambda/freq` hint, and whether to compute using CPU or upload to GPU worker.
-

Outputs

- `annularVector` — `Float32Array[K]` (per-frame): normalized per-annulus energy weighted by projected area.
- `motionDescriptor` — optional small vector (e.g., optical flow magnitude histogram or delta of `annularVector` across frames).
- `lowResDepth` — `Uint16Array` (`lowRes×lowRes`) & metadata `{z_near, z_far}`.
- `lowResNormals` — `Uint8Array` (oct-encoded, 2 bytes/pixel or 3 bytes/pixel) of size `lowRes×lowRes`.
- `frameRef` — storage handle to compressed frame blob (optional, WebP/Blob URL) or `null` if not stored.
- `psfParams` — simple optics metadata:
`{f_mm, f_number, sensor_px_size, lambda_um}`.

All outputs are serializable and optimized for IndexedDB storage.

Methods / API

The module exposes a compact worker-friendly API. Example in pseudocode:

```
// Create
const hf = HybridFresnelHarvester({ K:8, lowRes:64, quant:{depthBits:16} });

// Per-frame (eviction hook) -> non-blocking
await hf.processFrame(imageBitmap, { timestamp, frameId });

// Pull results (promises resolve when outputs ready)
const output = await hf.getLastOutput(frameId);
// output = { annularVector, lowResDepthBlob, lowResNormalsBlob, psfParams,
frameRef }

// Query a stream of annular vectors for online motion detection
hf.on('annular', (frameId, vec) => { /* visualize or classify */ });

// Flag frame for heavy reconstruction later
hf.flagForReconstruction(frameId);

// Shutdown / dispose
hf.dispose();
```

Worker model: `processFrame()` should be designed to hand off heavy work to a GPU or worker thread and return quickly. The main thread should only perform minimal `drawImage→OffscreenCanvas` steps.

Internal structure & flow

1. **Acquire pixel data** (fast path): draw `ImageBitmap` into `OffscreenCanvas` at native size or downsampled size, then either `getImageData()` or upload to GL texture.
2. **Precompute fast features**: compute luminance and simple temporal deltas; optionally compute a tiny thumbnail (8×8) for quick meta checks.
3. **Depth & normals pathway**:
4. If `depth` / `normals` supplied, accept and quantize directly.
5. Else run a *lightweight* depth estimator on a downsampled image (e.g., 64–128 px) **or** run a fast normal-from-depth network. This is the slowest step and should be optional / GPU-accelerated.
6. **Normals → annular bins**: convert normals to incidence angle $\theta = \arccos(n_z)$, map θ into K bins, compute $\text{annularVector}[k] = \sum (\text{lum} * \cos\theta)$ normalized by bin area.
7. **Normals → integrate → sag**: integrate gradients (Poisson) or directly reconstruct coarse depth; quantize depth to `Uint16` with `z_near/z_far` in metadata.
8. **Encode & emit**: oct-encode normals (2 bytes/pixel) and store `Uint16` depth buffer; emit `annularVector` immediately for downstream visualization.
9. **Finalize storage**: asynchronously write `lowResDepth`, `lowResNormals`, `annularVector`, `psfParams`, and optional compressed frame blob to IndexedDB.

Flow diagram (linear): `ImageBitmap -> OffscreenCanvas -> (fast features -> emit) + (lowRes depth/normals -> encode -> store) + (frameRef -> store)`

Data formats & quantization

- **Depth**: `Uint16` with linear mapping: `uint = round(clamp((z-z_near)/(z_far-z_near), 0, 1)*65535)`; store `z_near/z_far` in metadata.
 - **Normals**: octahedral encode into two `Uint8` channels (or `3×Uint8` for direct pack) — use well-known encode/decode functions.
 - **AnnularVector**: `Float32Array[K]`, normalized to `[0, 1]` or z-score across recent frames.
 - **Storage**: store blobs in IndexedDB as `ArrayBuffers`; avoid storing raw `ImageBitmaps` — store compressed frame blobs if needed.
-

Performance & resource notes

- **Latency target**: `processFrame` should not block >10–50ms on the main thread. Offload depth nets to worker/GPU.
 - **Memory**: lowRes buffers `64×64×2bytes = 8 KB` for depth; normals `64×64×2 = 8 KB` — tiny.
 - **Compute budget**: run full depth only on decimated frames (1 in N) or on frames flagged by annular motion spikes.
 - **Fallbacks**: when GPU/workers unavailable, compute only `annularVector` from thumbnails and skip depth.
-

Application examples

1. **Motion visualization (live)**: subscribe to `annular` events to animate zone energy and detect crossing events (digital PIR analog).

2. **Event triage:** use `annularVector` spikes to flag frames for reconstruction; later, reload high-quality frame and `lowResDepth` as seed for high-fidelity depth & PSF→superres pipeline.
3. **PSF-aware deblurring:** for flagged frames, compute `sag` from depth and construct phase Φ , produce space-variant PSFs and perform deconvolution or supervised SR.

Integration blueprint (IndexedDB schema)

Object store: `frames` Key: `frameId` Value:

```
{ ts, frameRef?, annularVector: Float32Array, lowResDepth: ArrayBuffer, lowResNormals: ArrayBuffer, psfParams }
```

Access patterns: - **Realtime:** query latest `annularVector` entries. - **Archive:** fetch by `frameId`, load `frameRef` blob, run reconstruction.

Example pseudocode: eviction hook

```
async function onEvict(frameId, imageBitmap) {
  // fast: draw thumbnail and compute annular vector
  const imData = await drawToOffscreenAndGetImageData(imageBitmap,
    {downscale: 128});
  const annular = computeAnnularFromImageData(imData, /*fast normals or proxy*/);
  hf.emit('annular', frameId, annular);

  // spawn worker job to compute lowRes depth/normals and store all artifacts
  hf.processFrame(imageBitmap, {frameId, ts:Date.now()});
}
```

Failure modes & mitigations

- **Compression artifacts:** store `lowResDepth` pre-eviction to avoid re-decode loss. Mitigate by computing depth on raw decoded frame.
 - **Noisy normals:** use temporal smoothing and confidence maps; store per-pixel confidence if available.
 - **Étendue misconceptions:** annular aggregation improves SNR but does not create new spatial detail — use it for detection, not for magic resolution gains.
-

Minimal implementation checklist (developer)

- [] Worker + OffscreenCanvas pipeline for ImageBitmap→ImageData.
- [] Annular bin calculator + emitter API.
- [] Lightweight depth/normal estimator (or adapter for precomputed inputs).
- [] Quantization & octahedral encode/decode utilities.

- [] IndexedDB storage layer and schema.
- [] Flagging API for deferred reconstruction.

License & notes

Keep this module MIT/BSD compatible. Keep code modular, testable, and GPU-friendly.

End of document.